

2.5

Which of the following equalities are always true?

$$\text{Twin}(\text{Twin}(e)) = e$$

$$\text{Next}(\text{Prev}(e)) = e$$

$$\text{Twin}(\text{Prev}(\text{Twin}(e))) = \text{Next}(e)$$

$$\text{IncidentFace}(e) = \text{IncidentFace}(\text{Next}(e))$$

$$\text{Twin}(\text{Twin}(e)) = e$$

1. Half edge e goes from some vertex u to vertex v . $u \rightarrow v$
2. $\text{Twin}(e)$ is the same edge as e but it is reversed. $v \rightarrow u$
3. $\text{Twin}(\text{Twin}(e))$ reverses the edge again. So edge goes back to $u \rightarrow v$
4. Therefore this equivalence is always true

$$\text{Next}(\text{Prev}(e)) = e$$

1. Take boundary cycle of the face $F = \text{IncidentFace}(e)$
2. Cyclic linked list, $\text{Prev}(e)$ is the edge that comes right before e
3. Then the $\text{Next}(\text{Prev}(e))$ function moves forward one step from the previous edge
4. This will then step back to the original edge e
 - Therefore this equivalence is always true because Next and Prev are inverses on the same boundary cycle.

$$\text{Twin}(\text{Prev}(\text{Twin}(e))) = \text{Next}(e)$$

1. Beginning with e , go to $\text{Twin}(e)$.
 - Now walking on other face across the edge, as $\text{IncidentFace}(\text{Twin}(e))$ is the face that is on the other side.
2. $\text{Prev}(\text{Twin}(e)) \rightarrow$ move one edge backward along the boundary of the face on the other side
3. $\text{Twin}(\text{Prev}(\text{Twin}(e))) \rightarrow$ Jump across the edge to its opposite half edge
4. $\text{Next}(e) \rightarrow$ Step forward one along boundary of the face on the left of e
 - The above operations are not always true because they are on different face cycles. $\text{Next}(e)$ stays on boundary of $\text{IncidentFace}(e)$ while the $\text{Prev}(\text{Twin}(e))$ walks along the boundary of the other face. In some cases these might be equivalent, but for most shapes these paths do not align.

IncidentFace(e) = IncidentFace(Next(e))

1. Next(e) is the next half-edge along the boundary of IncidentFace(e)
2. So, Next(e) lies on same face boundary as e
3. Therefore it must have the same incident face.

- This is proven true and this is always equivalent.

2.9

Suppose that a doubly-connected edge list of a connected subdivision is given. Give pseudocode for an algorithm that lists all faces with vertices that appear on the outer boundary.

1. Walk around the outer face boundary and mark all of the vertices that appear on the walk.
2. For each vertex, walk around all faces that are incident to that vertex and mark the faces. Use the vertex-rotation step to do this
3. Output all of the marked faces

Pseudocode:

ListFacesOuterBoundary(DCEL):

Get the outer face f_out that is unbounded, which is either stored or is infinity.

OuterVertex[v] = false for all vertices v

eFirst = f_out.OuterComponent

e = eFirst

Loop until e == eFirst:

v = Origin(e)

OuterVertex[v] = true

e = Next(e)

FaceMarks[x] = false for all faces x

for vertex v with OuterVertex[v] == true:

e0 = IncidentEdge(v)

if Origin(e0) != v:

e0 = Twin(e0)

e = e0

loop until e == e0:

FaceMarks[IncidentFace(e)] = true:

e = Twin(Prev(e))

for each face x:

if x != f_out and FaceMarks[x] == true:

output x

2.10

Let S be a subdivision of complexity n , and let P be a set of m points. Give a plane sweep algorithm that computes for every point in P in which face of S it is contained. Show that your algorithm runs in $O((n+m)\log(n+m))$ time.

1. Sweep a vertical line from left to right, need to initialize data structures and maintain during the algorithm.

- Create event queue Q with all subdivision vertices and all query points sorted by increasing x and breaking ties with increasing y
- Status tree T . Balanced BST storing exactly the edges that intersect the sweep line to the right of current event x .
- When processing query point $p = (x_p, y_p)$, look in T for edge above p at $x = x_p$ which is the ceiling boundary of the face that has p . Then use DCEL IncidentFace to indicate the side face

2. Process edges with a half-edge label for the left-right sweep.

- For each edge in the DCEL:
- Choose the version that has the origin as an endpoint with a smaller x (and if tied then a smaller y) and then call it $LR(e)$
- In the status tree, store the geometric segment and keep a pointer to the half edge $LR(e)$

3. Construct the event queue

- Put into the event queue Q all subdivision vertexes v and every query point $p \in P$

4. Status Tree Ordering

At sweep position $x = \text{currentX}$, every active segment intersects the sweep line at a y value. So active segment = s and it intersects sweep line at $y = ys(\text{currentX})$

- BST key is the intersection of the line at the y value

5. Process events in increasing order $\rightarrow$ Process all events in increasing (x, y)

5A. Let x_v be the x coordinate for the vertex event at subdivision vertex v

- Partition the incident edges into edges that start at v and edges that end at v by examining the left and right endpoints
- Update the status tree by deleting all of the edges that end at v and inserting all of the edges that start at v

5B: Need to get the edge immediately above p

- In the BST T , perform a successor query for y_p
- $E_above = \text{Successor}(T, y_p)$
- Multiple cases need to be considered.
- Nothing exists above, then p is the unbounded outer face
- Otherwise, call the stored half edge pointer of the segment/edge found above by

using the pre-defined pointer $h = LR(e_above)$

- Return the correct face using DCEL

- Since $IncidentFace(h)$ is face that sits to left of half-edge h and h is left to right, the left side is above the edge and the blow side is the twin's incident face

- Therefore, output $Face(p) = IncidentFace(Twin(h))$ as the output.

Runtime:

Sorting event queue of size $O(n + m) = O(N \log N)$

Subdivision edges being inserted and deleted once = $O(n)$ updates

Each query point does one search = $O(m)$ searches

Every BST op costs $O(\log N)$

$N = n + m$

Total Runtime = $O(N \log N) + O(n \log N) + O(m \log N) = O((n + m) \log(n + m))$

3.2

A rectilinear polygon is a simple polygon of which all edges are horizontal or vertical. Let P be a rectilinear polygon with n vertices. Give an example to show that $n/4$ cameras are sometimes necessary to guard it.

Many prongs polygon where more than one camera is needed to see two prongs. But this needs to be four prongs... $\rightarrow$ comblike rectangle

1. Construct a long rectangular hallway, and along its top side cut k deep and thin rectangular cuts.

2. With each rectangle there are 4 vertices. Each cut in the hallway is made by going up, then going right, then going down, and then going right back to the main boundary line.

- 4 new turns = 4 vertices per cut

- With 4 cuts, $n = 4 + 4k \rightarrow k = (n-4)/4 = (n/4)$

3. Going from the art gallery worst-case prongs example here.

- Each cut has a deep bottom corner, which is a reflex vertex pocket

- From any point in the polygon, line of sight into cut must pass through the narrow opening to the hallway

- Because cuts are separated by walls, camera that is positioned and supposed to observe the entirety of the deepness of the hallway has its view into other cuts in the hallway blocked by the vertical walls

- Therefore each of the cuts in the hallway require their own camera, as one camera cannot see into multiple cuts

- Better explained: No camera can see the two deepest points of two separate cuts of the hallway at once (deepest points in the crater of the comb in the rectangle).

4. Since $n = 4 + 4k$, $k = (n - 4)/4 = n/4$

- $n/4$ cameras are necessary for this example and show how sometimes they are necessary for rectilinear polygons.